

AI224: Operating Systems — Lecture 4

Dr. Karim Lounis

Jan - May 2025



Process Management

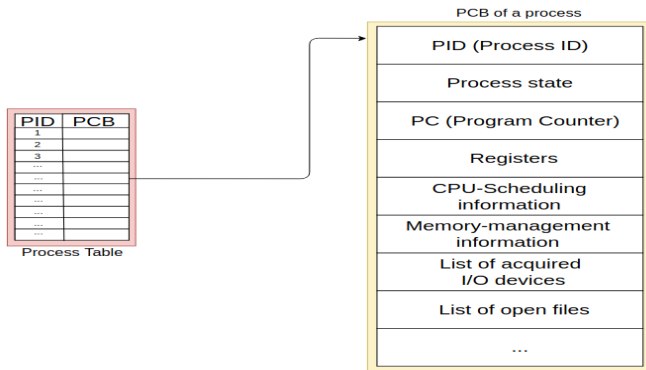
Process Management (Con't)

Context-switching

Context-switching, a.k.a., Process-switching

Recall: PCB (Process Control Block)

Every process is represented by a data structure called PCB (Process Control Block) that stores information about the process last state:



Enough information to resume execution and track process.

Context-switching

During its turnaround time, a process may obtain the CPU for a while, lose it for a while, then get it back to continue its execution.

Each time the process is interrupted (preempted), a certain number of information, called **context**, must be saved (state save) to the process PCB then loaded (state restore) again from it.

The context of a process is composed of:

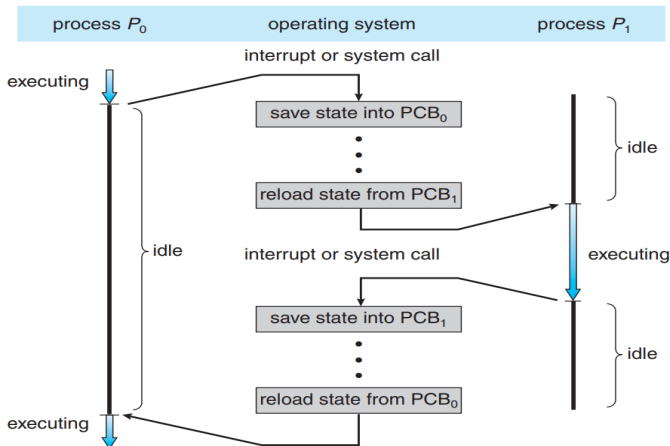
- The content of CPU registers, including the PC and the flags register.
- The memory-management information (i.e., address space).
- The acquired resources (e.g., files, network connections, I/O devices).
- ...

Context switching allows the kernel in one atomic operation to:

- 1 Save the context of current process into its PCB.
- 2 Load the context of the newly scheduled process from its PCB.

Context-switching (Con't)

The PCB serves as the repository for any information that may vary from process to process (Src: Textbook — Figure 3.9 — Page 109):



Context-switching (Con't)

Why context-switching is important for time-sharing systems:

- 1 It increases the CPU throughput.
- 2 It improves the CPU response time.
- 3 It prevents certain processes from hogging on the CPU infinitely.
- 4 It allows high-priority processes to be executed.
- 5 It reduces process turn-around-time.

However, context-switching has also a cost:

- 1 It depends on the size of the context.
- 2 The system must not be interrupted for a long time.

Example Context-switching (Con't)

Interrupt with context-switching

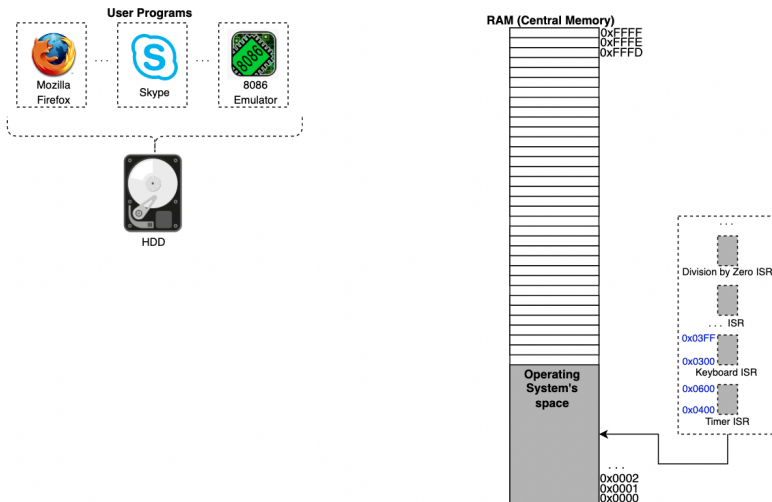
Looking at the various existing computer architectures, you would find out that the steps for context-switching may vary from one architecture to another — old architectures to modern architectures. I.e, there is no standard steps that all computers apply. In this course, we present the commonly used steps in the literature (for the sake of education).

Interrupt with Context-switching

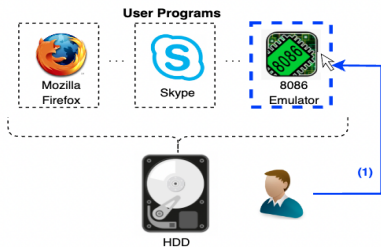
Assuming that process P1 is executing and process P2 is the next process in the ready queue. Then, if an interrupt occurs:

- ① CPU pushes the current PC onto the stack and updates the PC-register to contain the address (obtained from the Interrupt Controller) of the 1st instruction in the corresponding interrupt handler.
- ② The interrupt handler (a.k.a., ISR) starts execution:
 - It pushes all remaining registers onto the stack then performs its task (e.g., display error message, update clock, move mouse, etc).
 - It Invokes the scheduler (to cause context-switching), which:
 - Copies all registers values from the stack into PCB1.
 - Move PCB1 to end of ready/blocked queue, and PCB2 into the head.
 - It copies registers values from PCB2 into CPU-registers, except the PC values which get copied to the top of the new stack (P2's stack).
 - ISR executes RTI (ReTurn from Interrupt) causing CPU to automatically pop the PC value from the new stack to CPU's PC register.
- ③ The CPU then, starts/resumes the execution of process P2.

Example of Interrupt with Context-switching

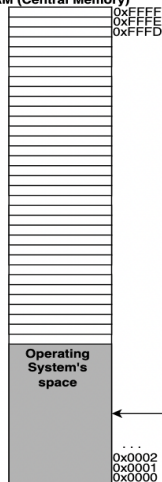


Example of Interrupt with Context-switching (Con't)

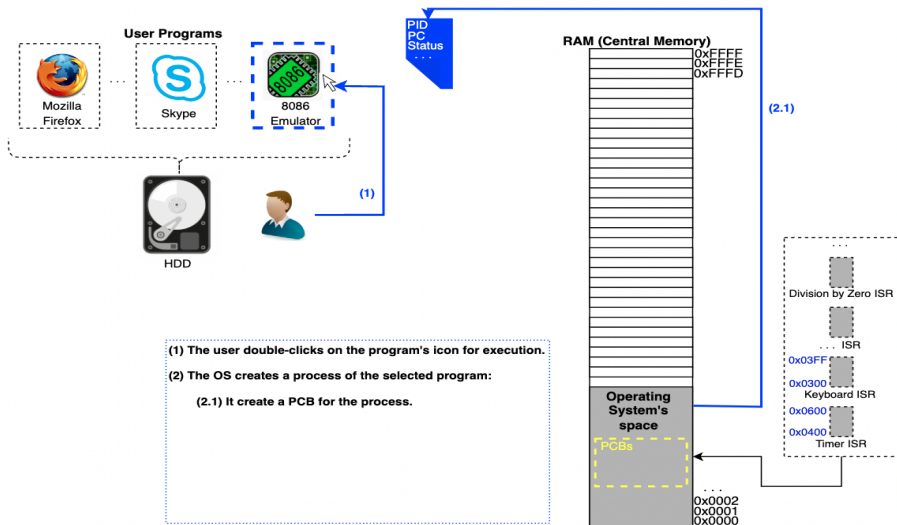


(1) The user double-clicks on the program's icon for execution.

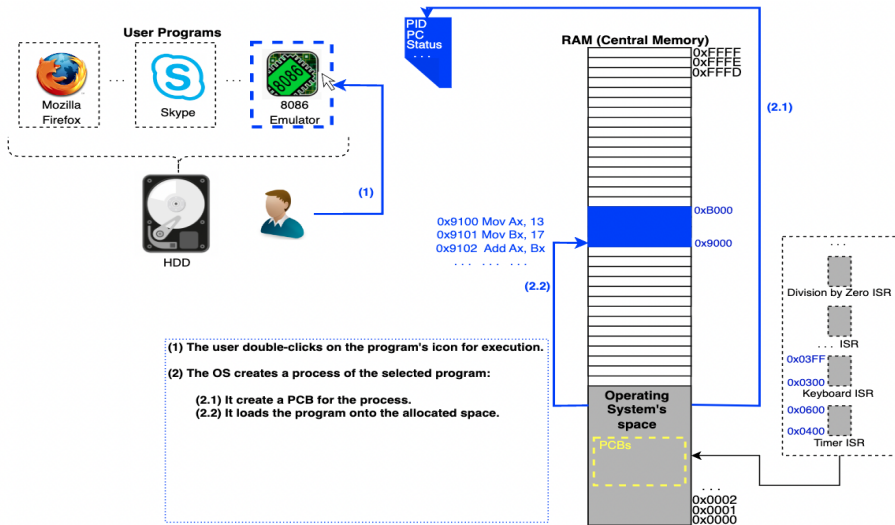
RAM (Central Memory)



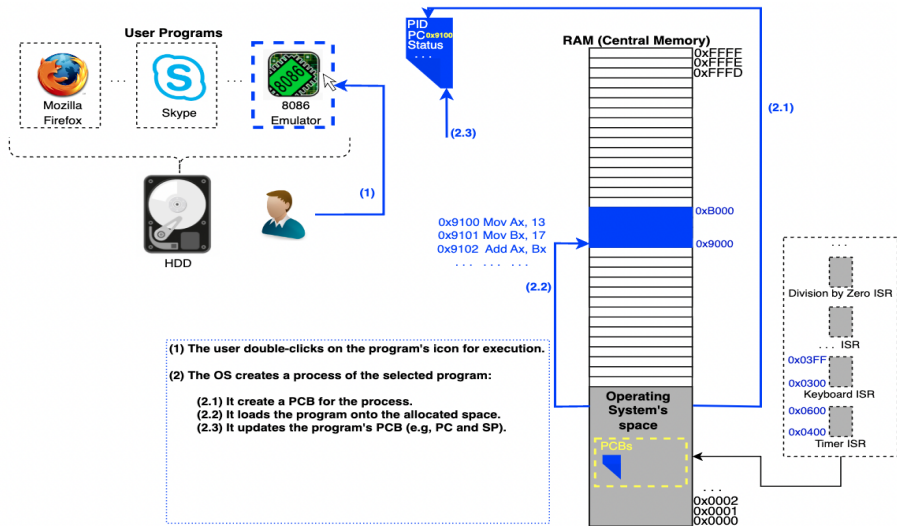
Example of Interrupt with Context-switching (Con't)



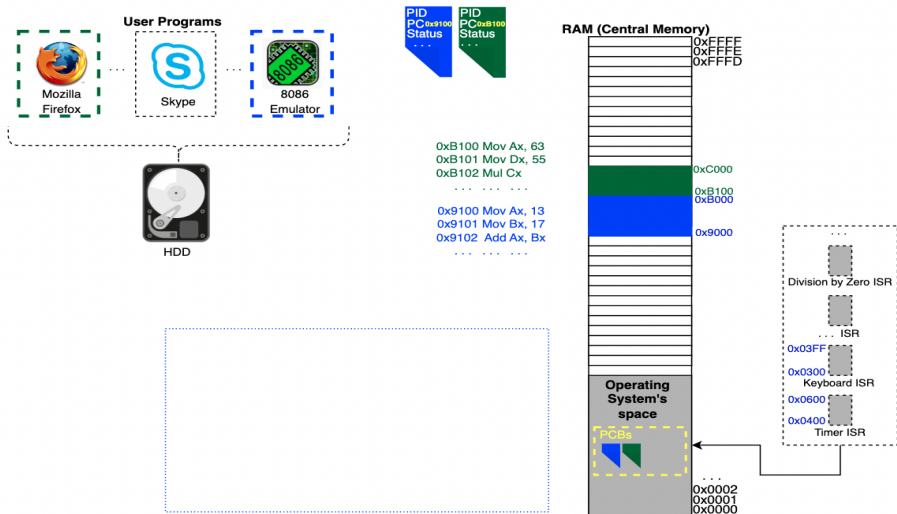
Example of Interrupt with Context-switching (Con't)



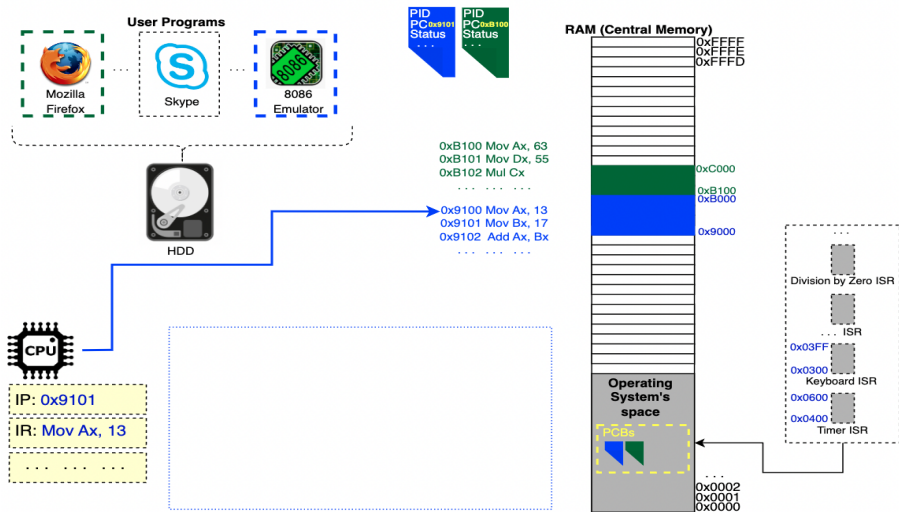
Example of Interrupt with Context-switching (Con't)



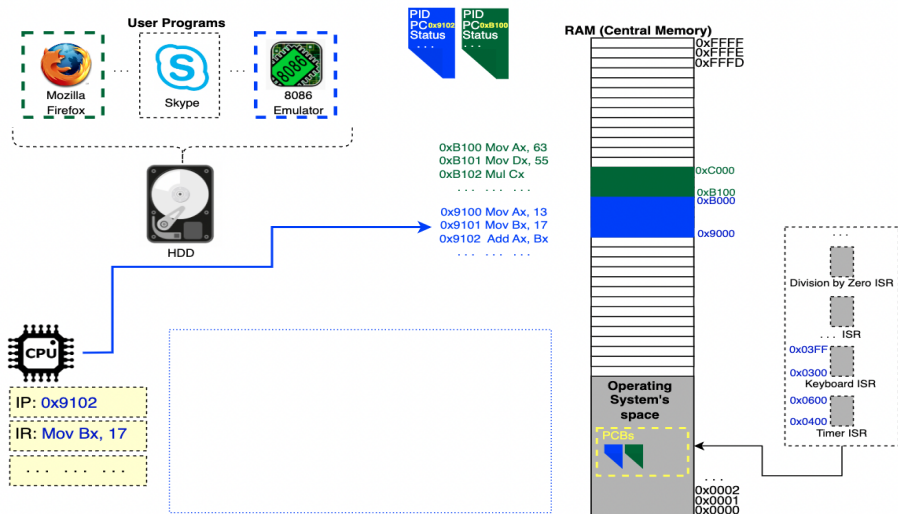
Example of Interrupt with Context-switching (Con't)



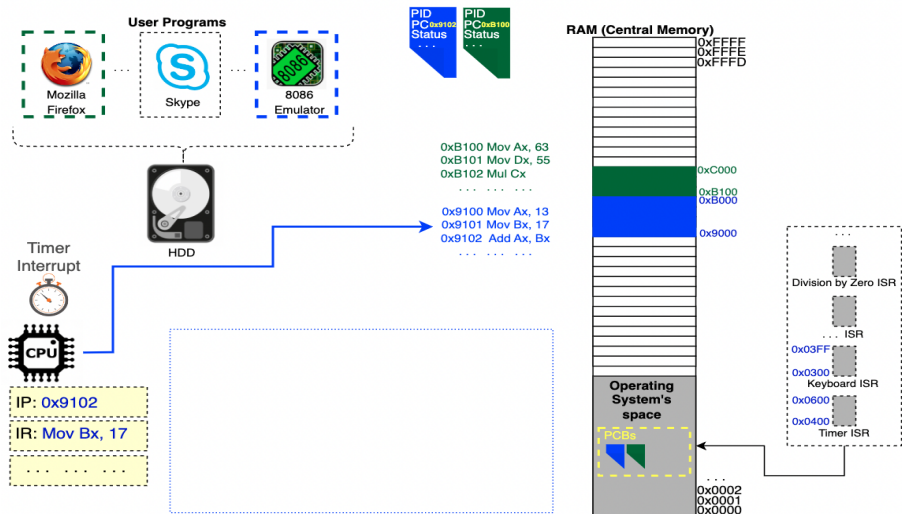
Example of Interrupt with Context-switching (Con't)



Example of Interrupt with Context-switching (Con't)

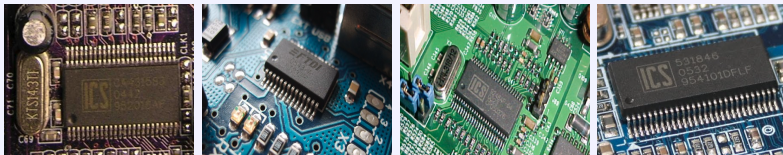


Example of Interrupt with Context-switching (Con't)



Example: timer

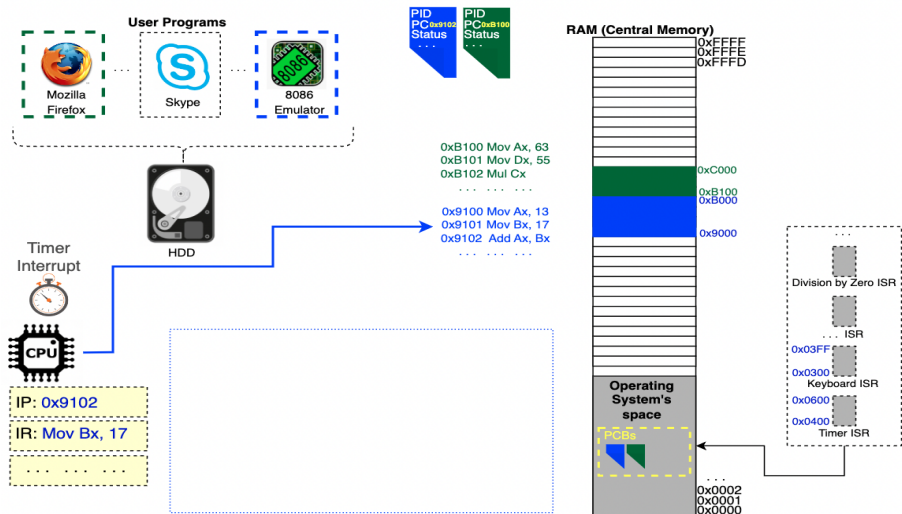
Timer. Is a special hardware that counts at a steady rate, like a clock. It can be configured to generate an interrupt (a signal to the processor) after a specific interval of time has passed.



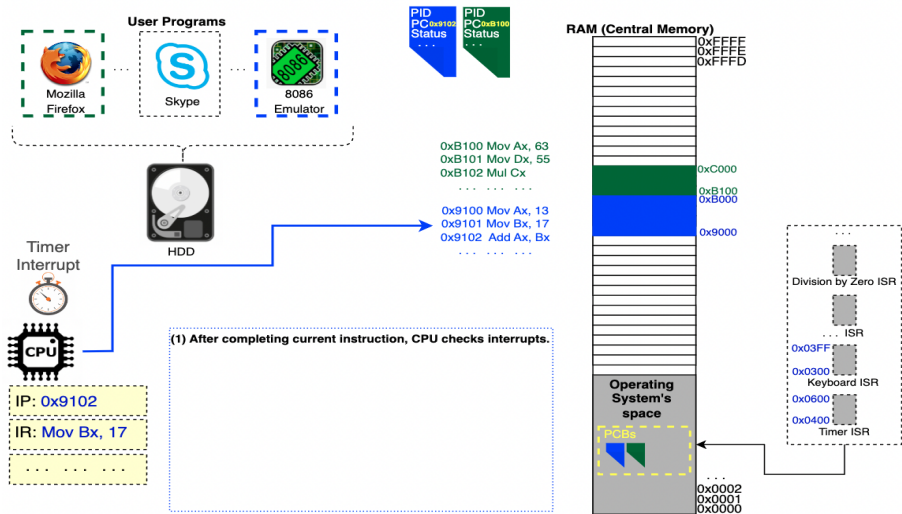
When timer reaches preset count or interval, it raises an interrupt signal to the processor, which acknowledges the interrupt and starts the handler execution. The latter:

- Update the system clock.
- Trigger a specific event (e.g., wake-up sleeping process, accounting — CPU utilization, idling the system, etc).
- Invoke the scheduler.
- Resumes interrupted process.

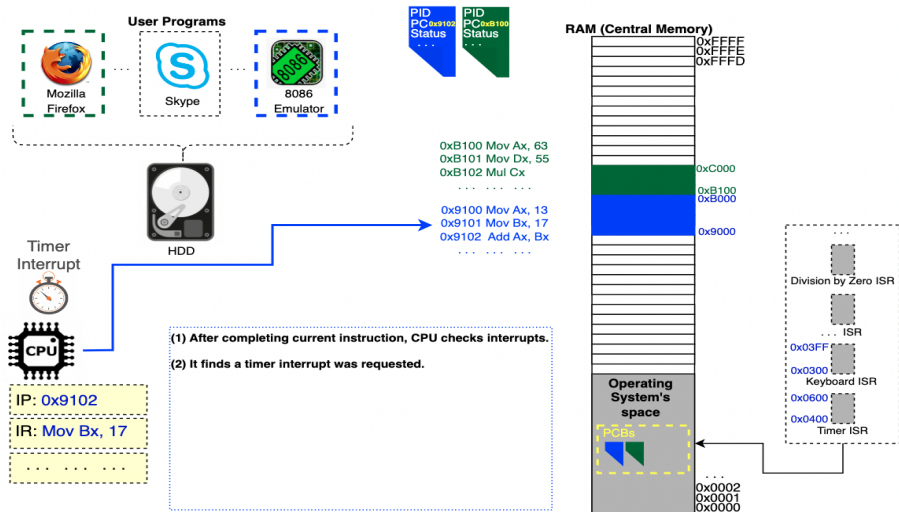
Example of Interrupt with Context-switching (Con't)



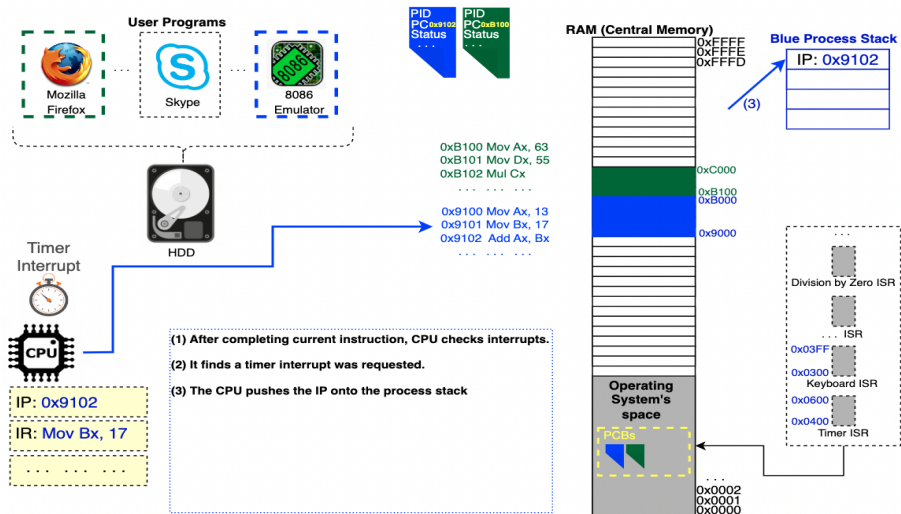
Example of Interrupt with Context-switching (Con't)



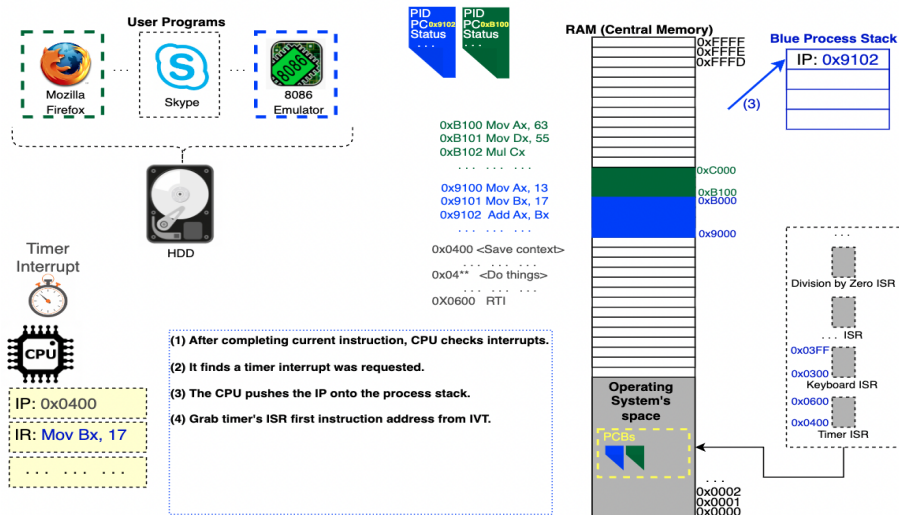
Example of Interrupt with Context-switching (Con't)



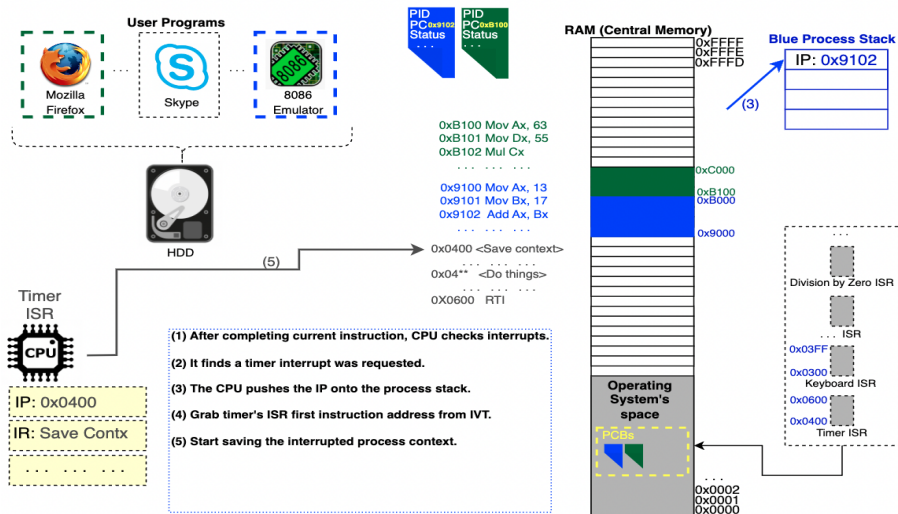
Example of Interrupt with Context-switching (Con't)



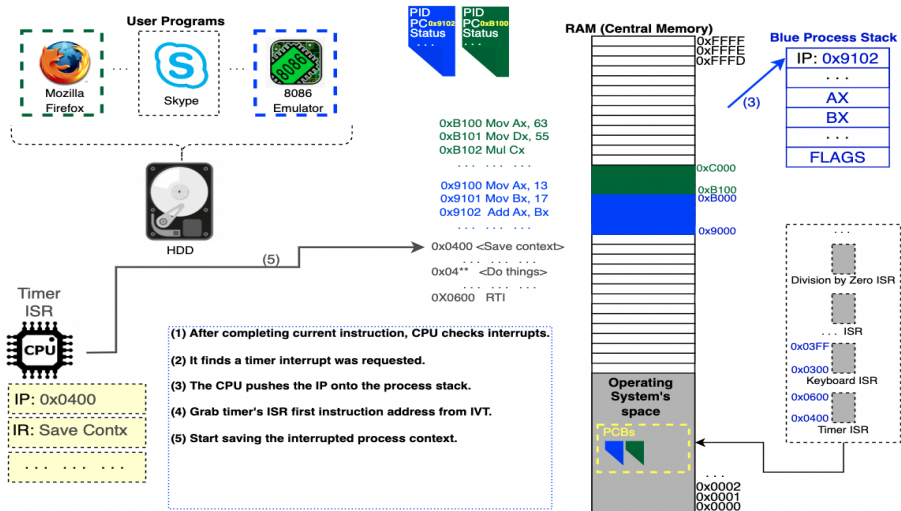
Example of Interrupt with Context-switching (Con't)



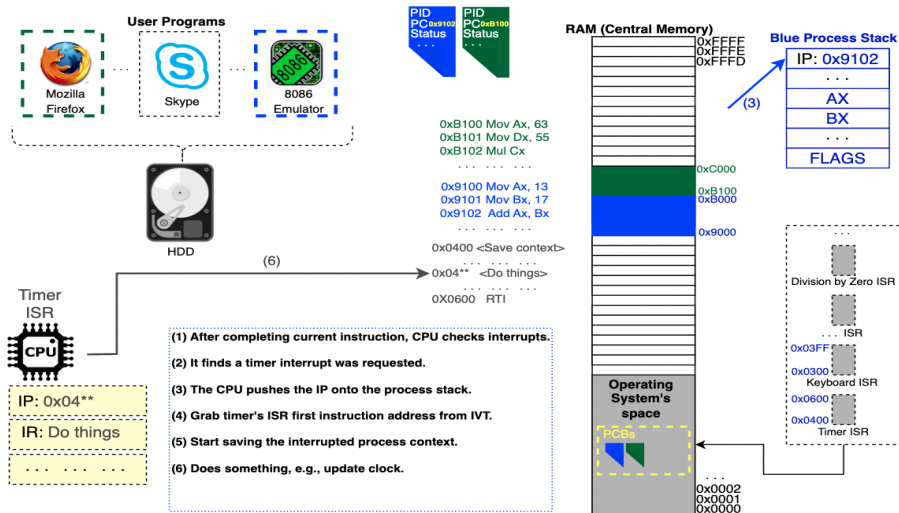
Example of Interrupt with Context-switching (Con't)



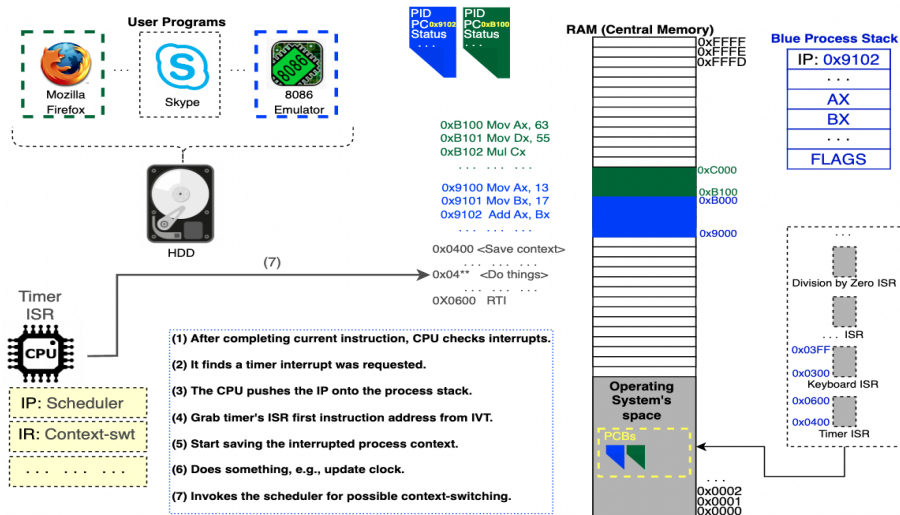
Example of Interrupt with Context-switching (Con't)



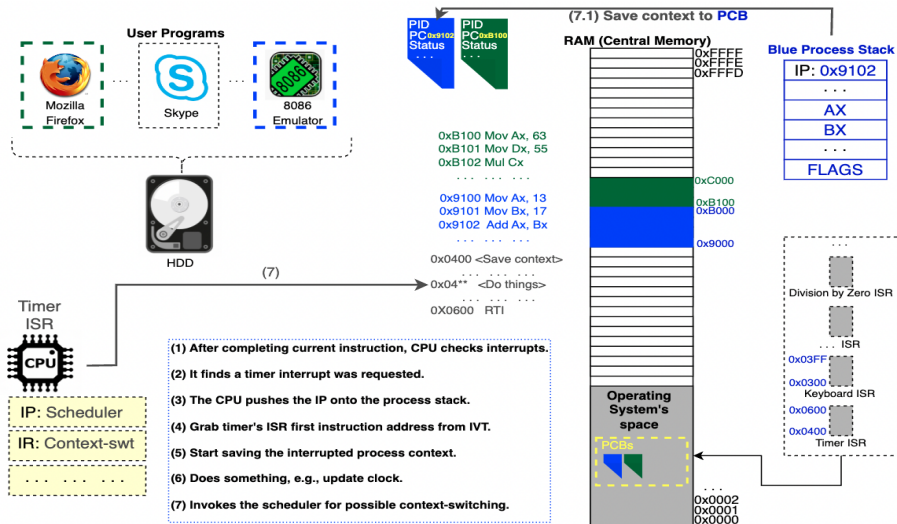
Example of Interrupt with Context-switching (Con't)



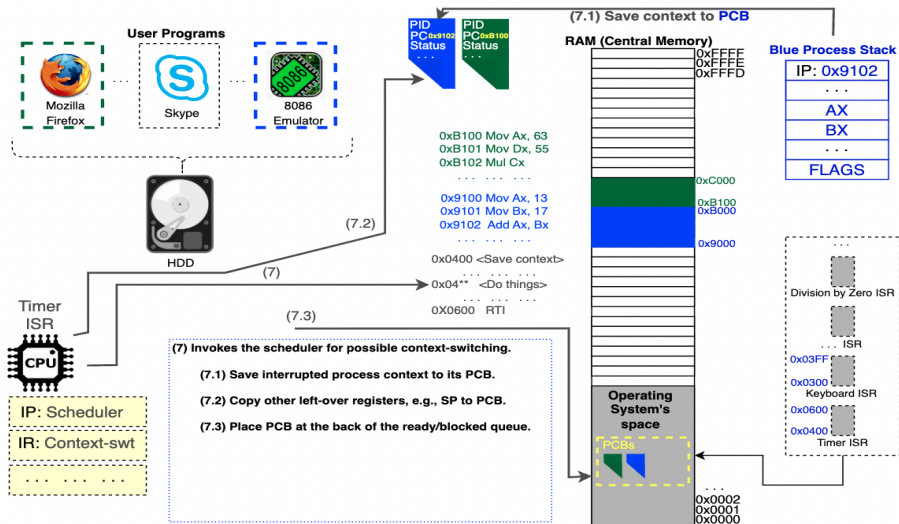
Example of Interrupt with Context-switching (Con't)



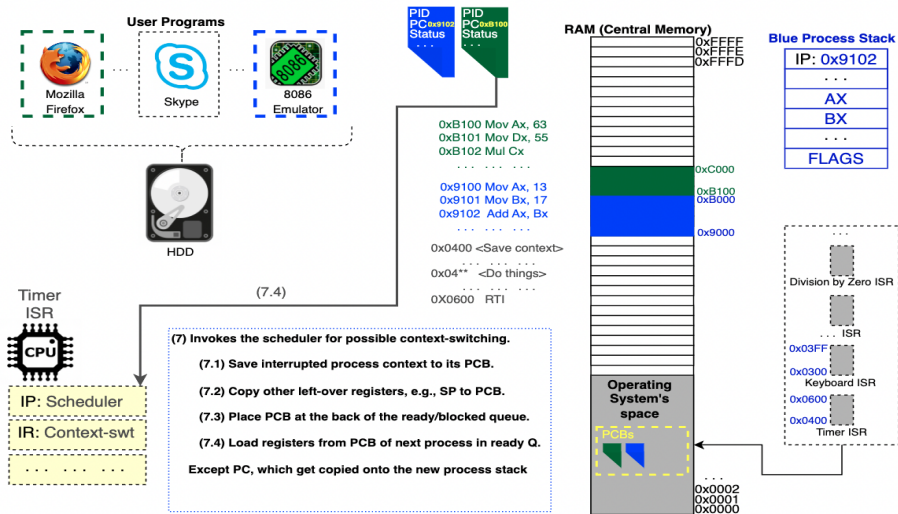
Example of Interrupt with Context-switching (Con't)



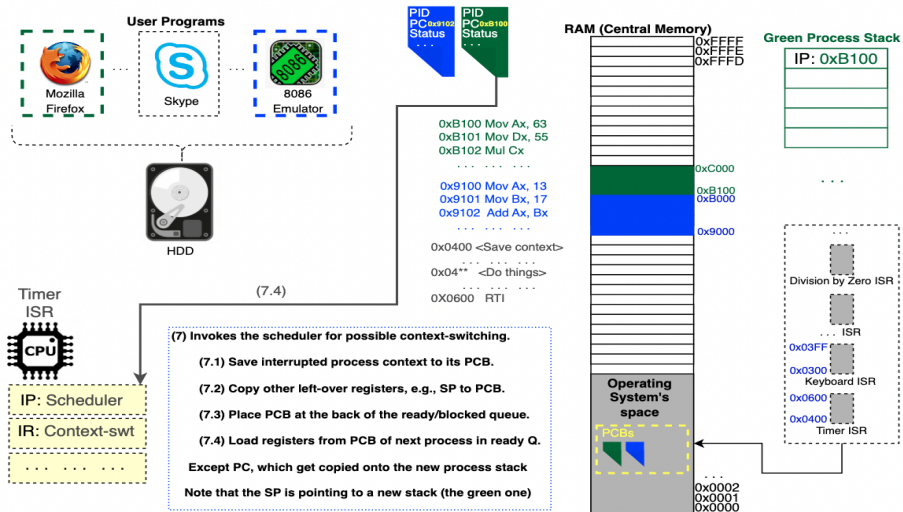
Example of Interrupt with Context-switching (Con't)



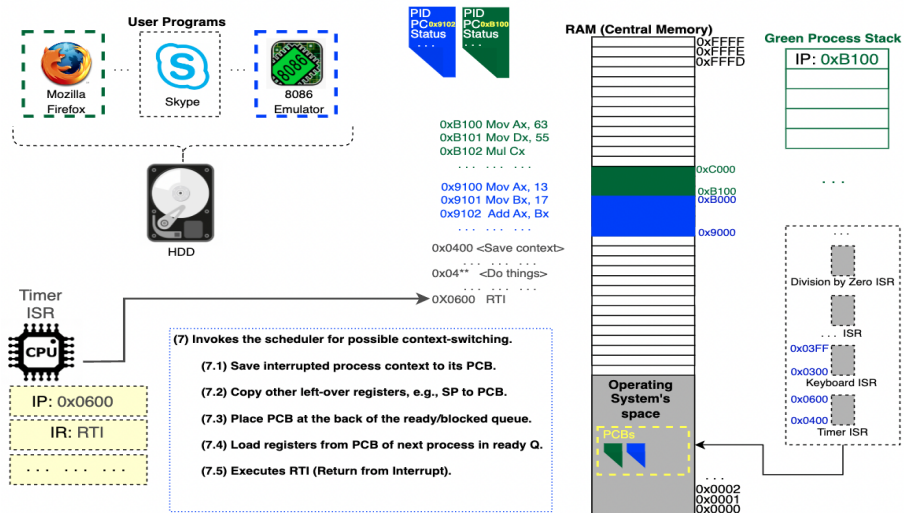
Example of Interrupt with Context-switching (Con't)



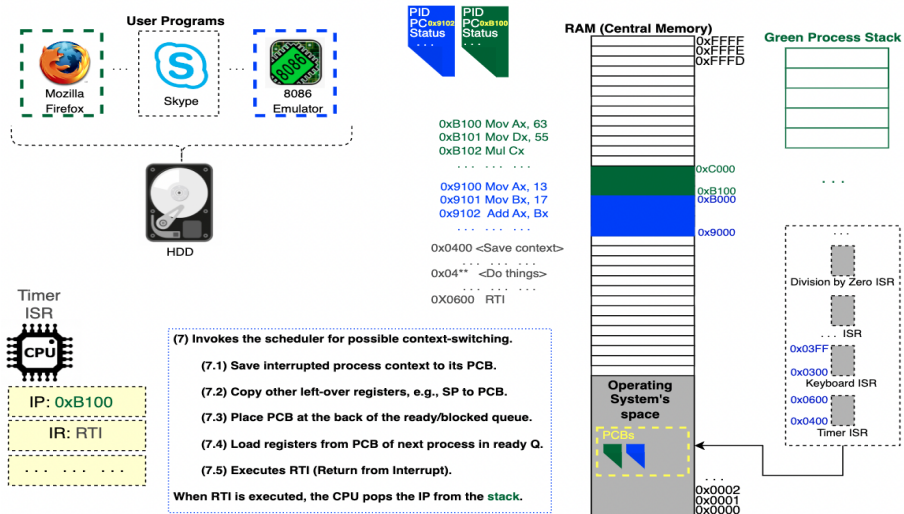
Example of Interrupt with Context-switching (Con't)



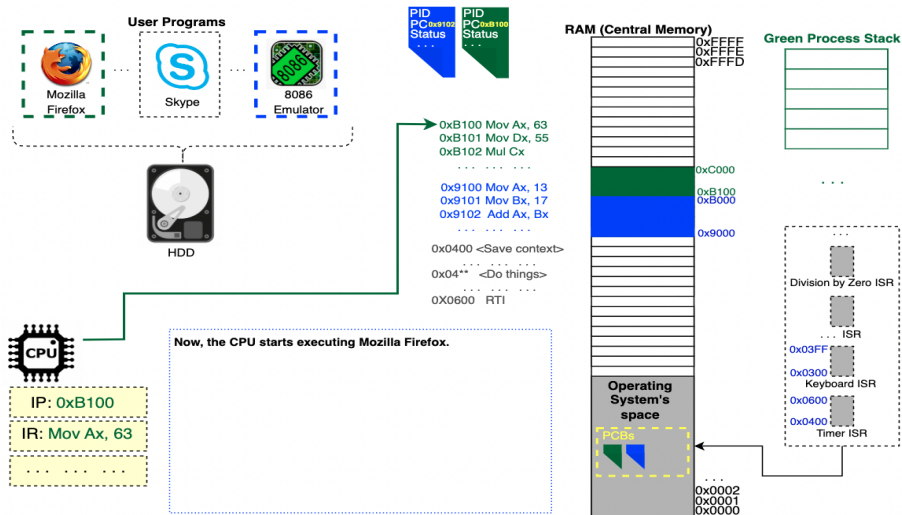
Example of Interrupt with Context-switching (Con't)



Example of Interrupt with Context-switching (Con't)



Example of Interrupt with Context-switching (Con't)



Interrupts without Context-switching

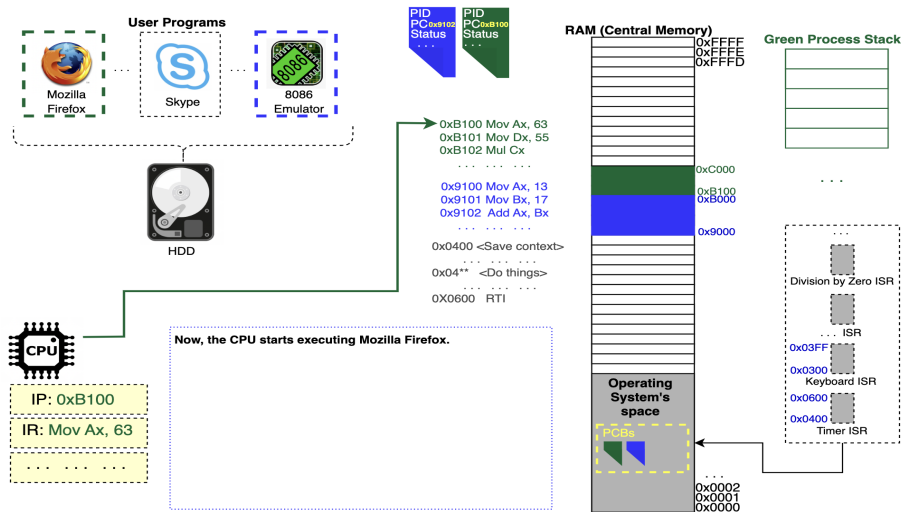
Interrupts without context-switching

Interrupts without Context-switching

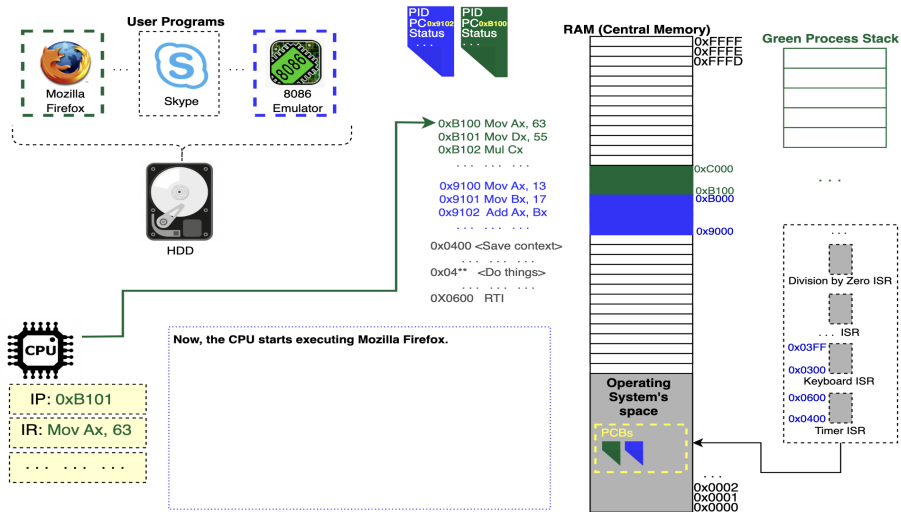
Assuming that process P1 is executing and a keyboard/mouse interrupt occurs:

- 1 CPU (hardware) pushes the current PC onto the stack and updates the PC-register to contain the address of the 1st instruction in the corresponding interrupt handler.
- 2 The interrupt handler starts execution by pushing all other registers onto the stack.
- 3 Performs its specific task that modifies the CPU registers content.
- 4 Pops back the registers values from the stack into the CPU.
- 5 The handler executes RTI (ReTurn from Interrupt) which causes the CPU (hardware) to pop PC value from stack to CPU's PC register.
- 6 The CPU then, resumes executing process P1.

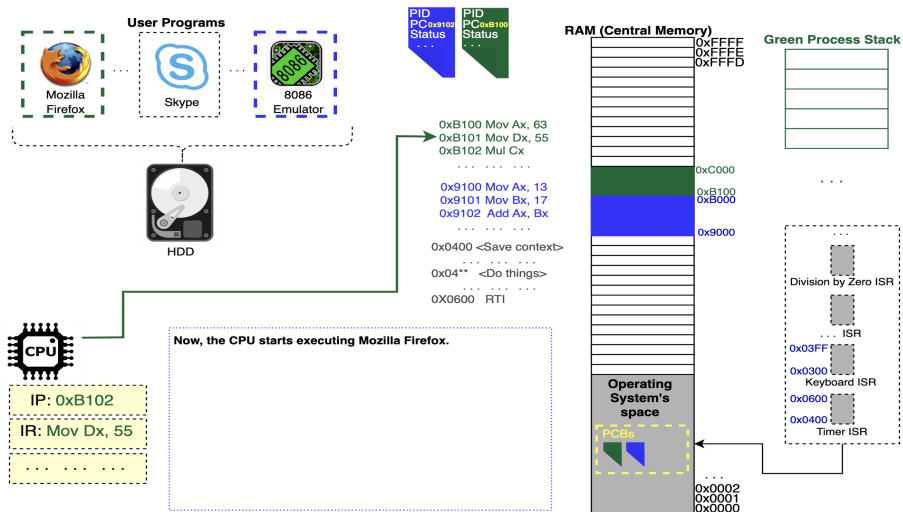
Example of Interrupt without Context-switching (Con't)



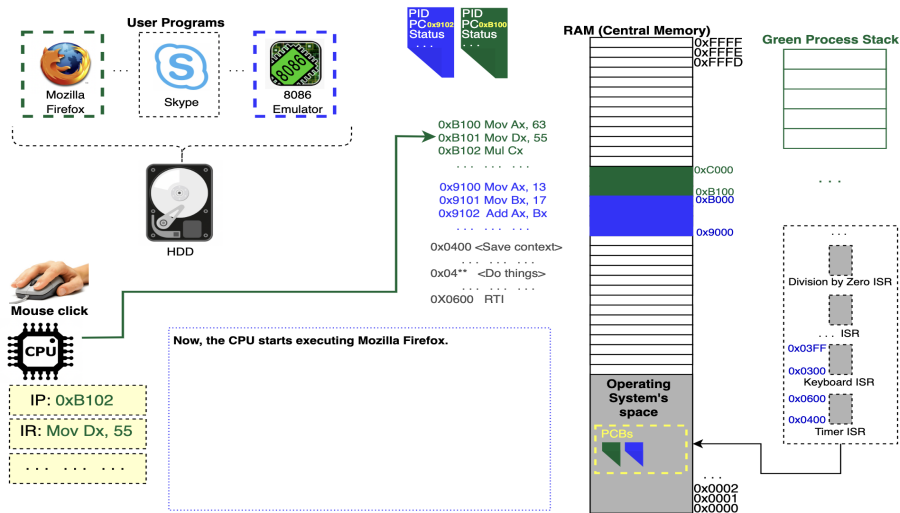
Example of Interrupt without Context-switching (Con't)



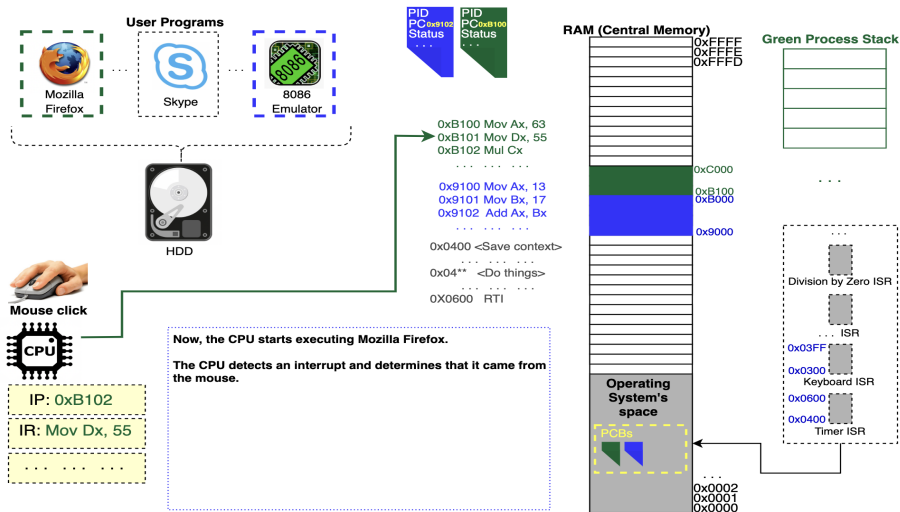
Example of Interrupt without Context-switching (Con't)



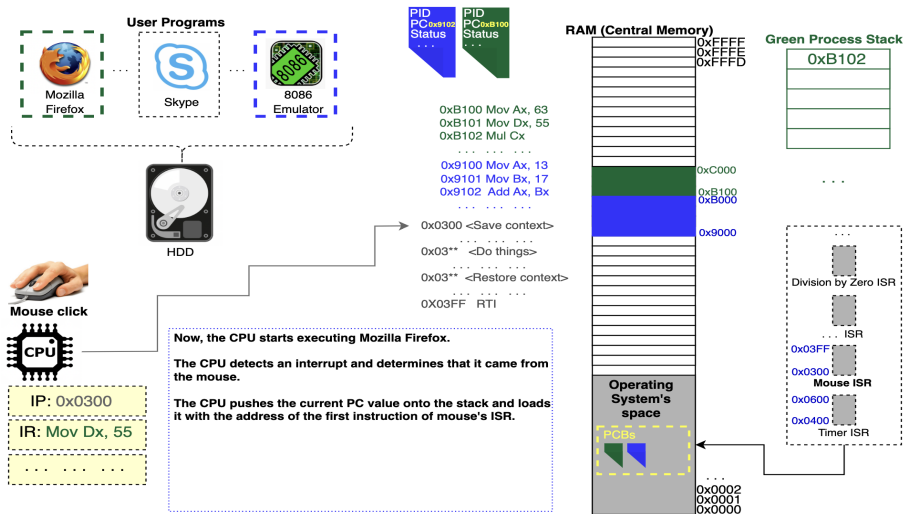
Example of Interrupt without Context-switching (Con't)



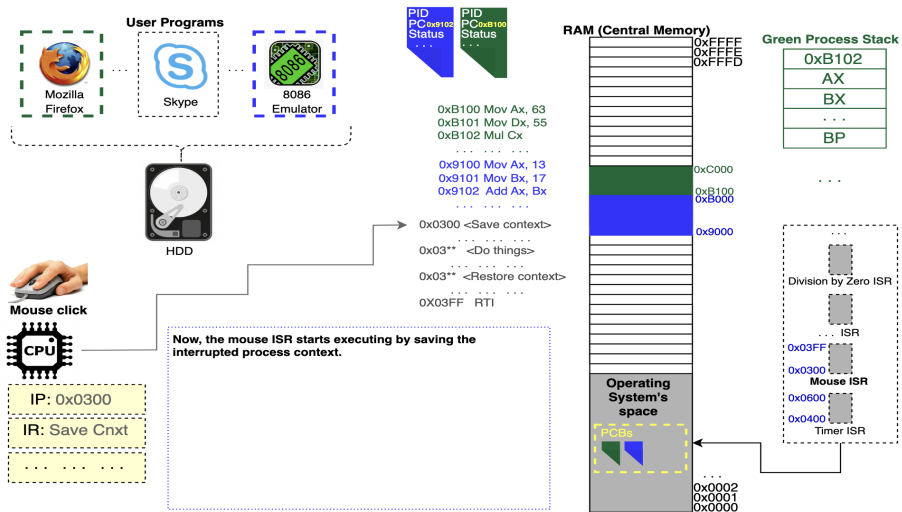
Example of Interrupt without Context-switching (Con't)



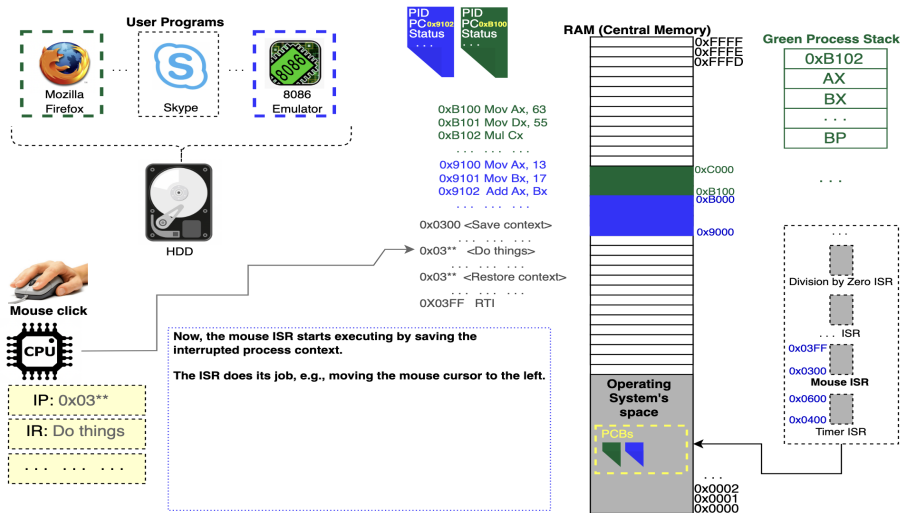
Example of Interrupt without Context-switching (Con't)



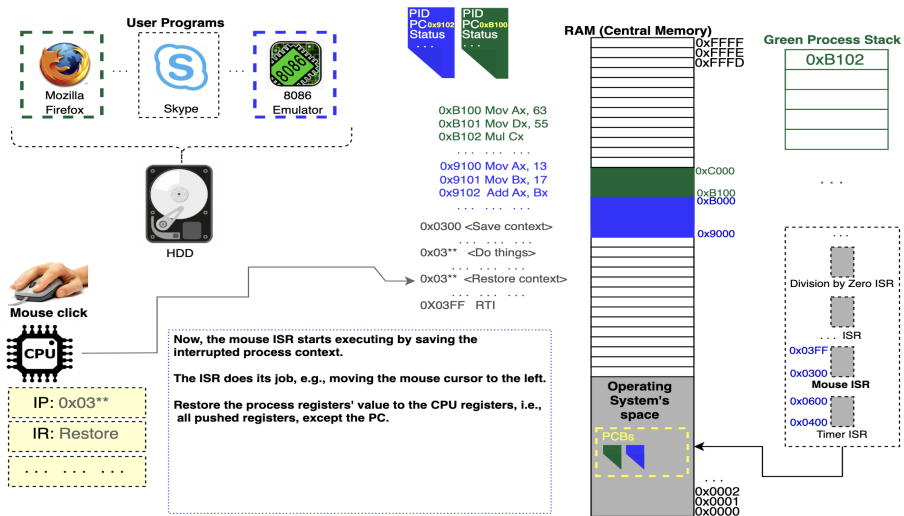
Example of Interrupt without Context-switching (Con't)



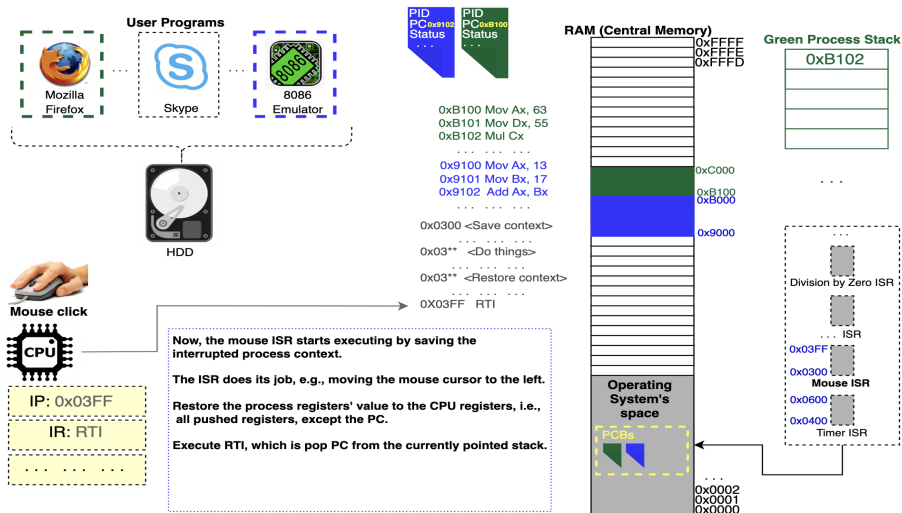
Example of Interrupt without Context-switching (Con't)



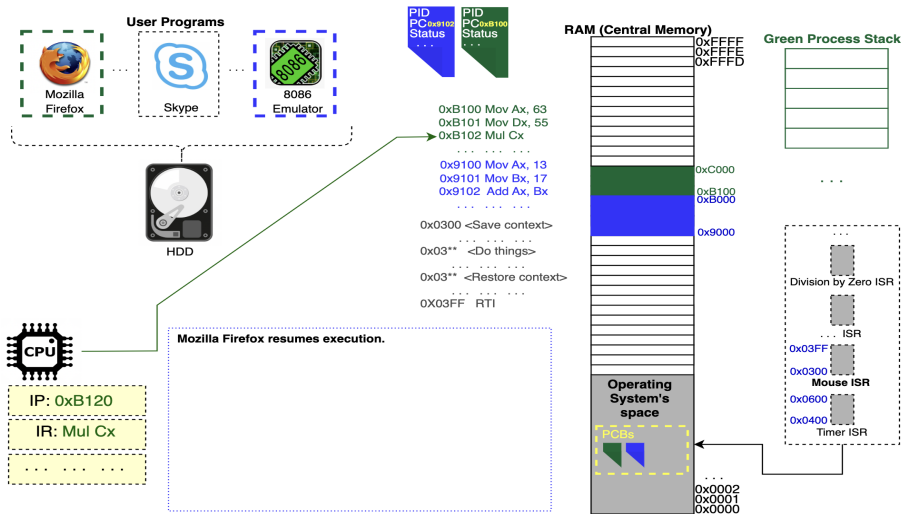
Example of Interrupt without Context-switching (Con't)



Example of Interrupt without Context-switching (Con't)



Example of Interrupt without Context-switching (Con't)



Interrupt Handler and Context-switching

The occurrence of an interrupt does not always imply a context-switch:

- 1 Certain keyboard/mouse interrupts.
- 2 DMA completion interrupts.
- 3 Raster interrupts.

Other interrupts cause a context switch:

- 1 System calls.
- 2 Exceptions.
- 3 Timer interrupt.

Of course, if there is only one process, then there is no context-switching even with timer interrupt.

Interrupt Handler and Context-switching

Generally, right after an interrupt (hardware or software):

- 1 The CPU pushes the PC value into the stack pointed by SP.
- 2 The CPU loads the PC register with a new value (@ of the handler).
- 3 The handler saves the remaining registers onto the stack.
- 4 The handler executes its specific task.
- 5 The handler determines whether a context-switching is needed or not.

If yes, a context switching is performed. The previously pushed registers are saved into the PCB (of the interrupted process) and the CPU is loaded with a new content, except the PC, from a new PCB.

If no, goto 6.

- 6 The handler Executes RTI.
- 7 The CPU pops back the PC value from the stack pointed by SP.

Interrupts Interrupt Interrupt Handlers

During the execution of an interrupt handler, other interrupts can occur.

Interrupts are executed in a nested order, i.e., if an interrupt of a higher priority occurs while another interrupt handler of a lower priority is executing, the current (low-priority) interrupt handler is interrupted.

When an interrupt handler is interrupted, its PC value is pushed by the CPU onto the stack dedicated for interrupts (located in kernel space).

The new handler saves the other register into kernel interrupt stack.

The kernel interrupt stack is pointed by ISP (Interrupts Stack Pointer).

Once the new handler terminates, it pops the registers of the old handler.

Executes RTI, which informs the CPU to pop the PC value from interrupt stack so that the old handler can resume execution.

Threads

Threads — Lightweight Processes

Threads

Definition

Thread: It is a lightweight process.

Lightweight: Does not need to allocate additional memory space.

- The process that creates threads shares its code, data, heap, resources (e.g., open files), user/group ID, ... etc, with the created threads.
- Threads only require dedicated stack and will have their proper values when they use the CPU registers.
- Thread context-switching is very fast.

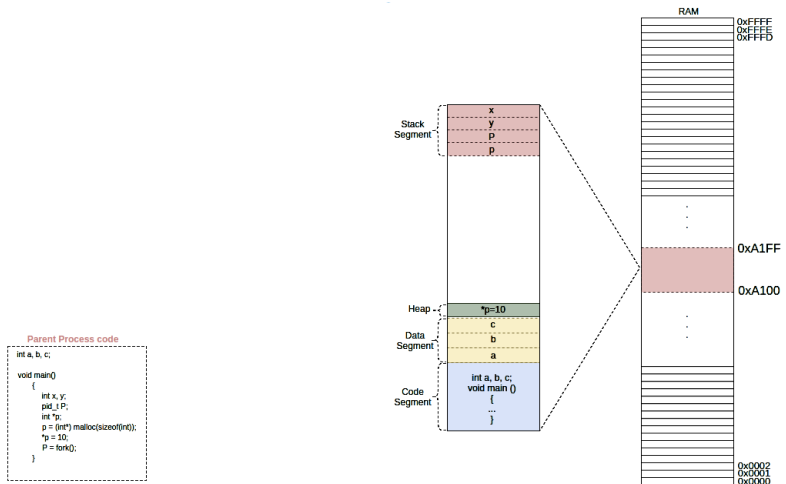
Similar to processes:

- Threads are identified by a TID (Thread Identifier).
- Each thread is associated to a data structure called TCB (Thread Control Block) which contains the thread's Identifier, thread's state, priority, thread's registers, and a pointer to the original process PCB.

E.g., In GNU/Linux, the command `$top -H -p 4234` would display the threads that are related to the process whose PID is 4234

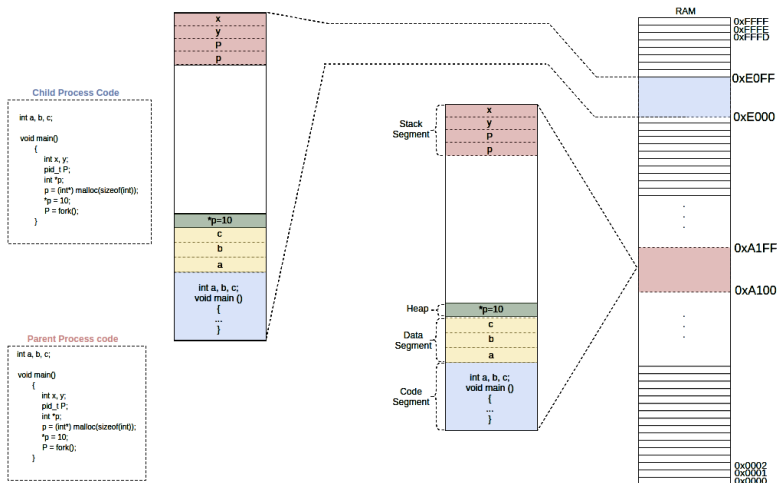
Threads vs Processes

Assuming you have a process that creates another process (child process) by invoking the system call `fork()`:



Threads vs Processes

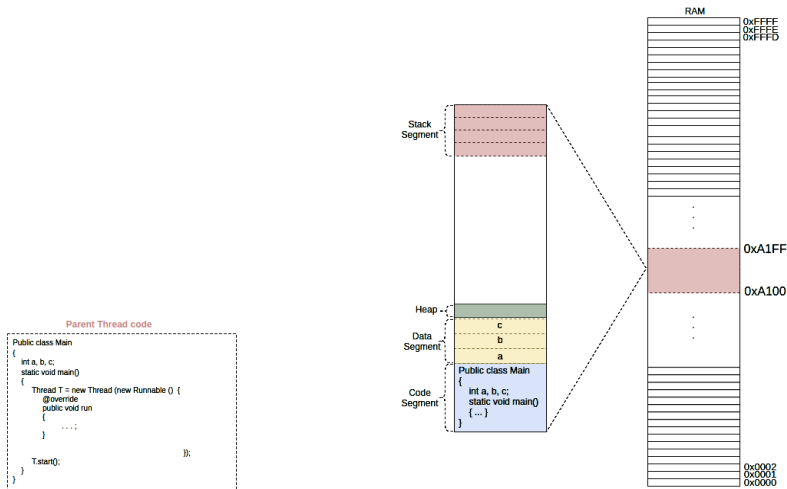
By executing `fork()` the OS duplicates the PCB, allocates a new memory space, and updates the PCB data fields:



Threads vs Processes

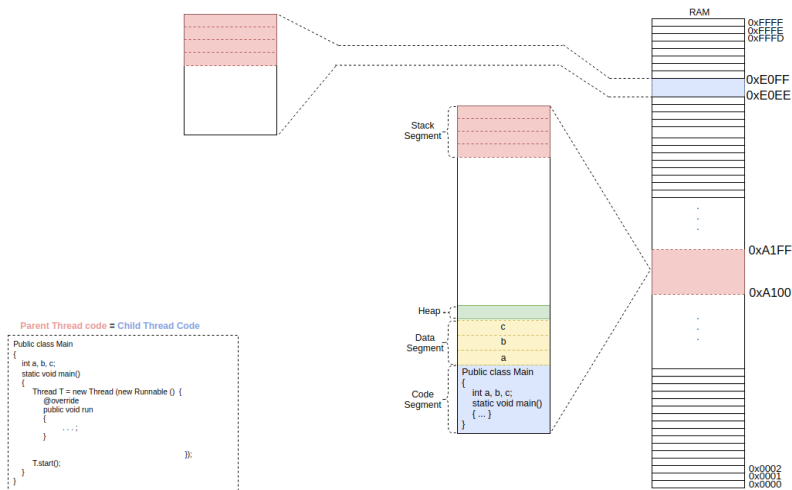
Assuming you have a process that creates a thread:

```
Thread T = new Thread(); T.start();
```



Threads vs Processes

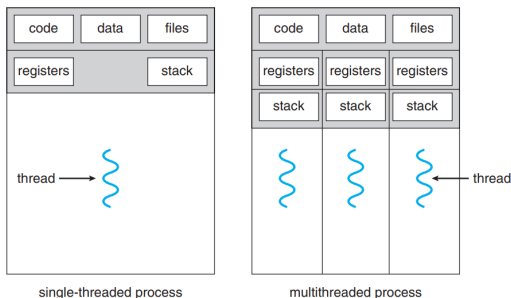
By executing the statements `Thread T = new Thread(); T.start();`



Threads

Older process model used to be composed of only one thread (single flow of execution). We talked about **single-threaded processes**.

Modern processes are **multi-threaded processes**, i.e., the process can perform more than one task at a time (multiple flows of execution).



⊖ New challenges for system designers and application programmers.

Threads

Most modern OS's kernel are now multithreaded, e.g., GNU/Linux use a kernel thread for managing the amount of free memory in the system.

Advantages of Threads

- **Responsiveness:** If a thread is blocked, the other one can continue running.
- **Resource sharing:** No need for interprocess communication. Threads from a same process would share code and data.
- **Economy:** We don't need to allocate a separate memory space.
- **Scalability:** Utilization of multiprocessors/multi-cores, whereas a single-threaded process uses only one core.

Executing a multi-threaded application on a single-core CPU system is meaningless (fake concurrency $\neq$ (real concurrency = parallelism)).

Modern CPUs can run more than 1 thread per CPU core (hyperthreaded processors) e.g., Intel i5-2410M, has 4 cores, runs 2 threads per core.

Threads (Example of Intel's hyperthreading)

You can see how many threads your CPU is able to execute at a time:

```
root@Human-Device:~# lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                 4
On-line CPU(s) list:   0-3
Thread(s) per core:    2
Core(s) per socket:    2
Socket(s):              1
NUMA node(s):          1
Vendor ID:              GenuineIntel
CPU family:             6
Model:                  42
Model name:             Intel(R) Core(TM) i5-2410M CPU @ 2.30GHz
Stepping:               7
CPU MHz:                798.231
CPU max MHz:            2900.0000
CPU min MHz:            800.0000
BogoMIPS:               4589.81
Virtualization:         VT-x
L1d cache:              32K
L1i cache:              32K
L2 cache:               256K
L3 cache:               3072K
NUMA node0 CPU(s):      0-3
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
                        pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx rdtscp lm constant_tsc
                        arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aperfmperf pni pclmulqd
                        q dtes64 monitor ds_cpl vmx est tm2 ssse3 cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic popcn
                        t tsc_deadline_timer aes xsave avx lahf_lm epb pti ssbd ibrs ibpb stibp tpr_shadow vnmi
                        flexpriority ept vpid xsaveopt dtherm ida arat pln pts flush_lld
```


Threads Scheduling

Traditionally, we use to work with:

- Non-multithreaded operating systems.
- Processes were the basic unit of CPU utilization.
- Scheduler deals with processes.
- Full context-switching.

Nowadays, we work with:

- Thread-aware operating systems.
- Threads are the basic unit of CPU utilization.
- Scheduler deals with threads.
- Full and partial context-switching.

Threads Levels

Threads are often implemented at two levels:

1. User-level Threads. In this type of threads, the OS is not aware of the presence of multiple threads in one process (Many-to-One threading model):

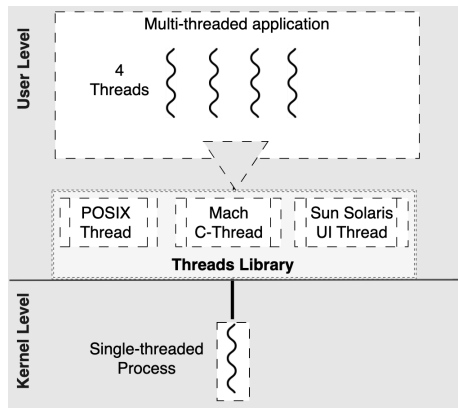
A threading library is used to perform threads context-switching.

Process memory will be used for threads stack allocation.

System calls from one thread block the main process .

The OS schedules the process as a single-threaded process entity.

Internal thread scheduling (lighter, flexible, and portable).



Threads Levels

Threads are often implemented at two levels:

2. Kernel-level Threads. In this type of threads, the OS is aware of the concept of threads (i.e., kernel is multi-threaded) and offers system calls to create and manages threads (One-to-One threading model):

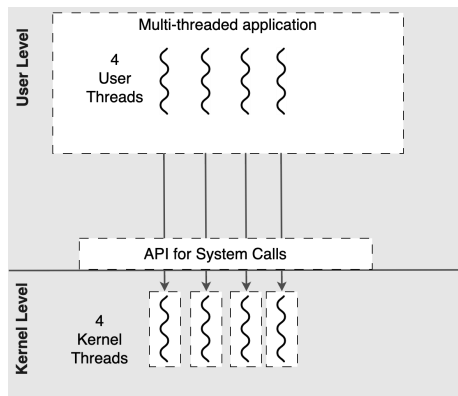
System calls are used to create physical threads.

Each user thread is mapped to one kernel thread.

System calls from one thread do not block the other threads .

Use multiple CPU cores at a time.

E.g., Pthreads API, Win32 API, and Grand Central Dispatch framework.



DMA (Direct Memory Access)

DMA — Direct Memory Access

DMA (Direct Memory Access)

Definition

Direct memory access (DMA) is the process of transferring data without the involvement of the processor itself. It is often used for transferring data from the memory to I/O devices or from I/O devices to the memory.

This is performed through the use of a special-purpose processor known as DMA controller, which notifies the CPU that it is ready for data transfer.

When the CPU is notified by a DMA, it relinquishes control of its external memory bus and grants the control of the bus to the DMA controller.

The DMA controller then transfers the specified amount of data and signals the processor upon completion of the transfer (i.e., DMA I/O completion).

Edmund Lai PhD, BEng, in Practical Digital Signal Processing, 2003 (cf., 9.2.2.4).

DMA (Direct Memory Access)

Each I/O device has a dedicated device controller, which is a hardware component that interfaces b/w the device and the computer.

Each I/O device is connected to a computer via a port (or slot) (e.g., USB, RJ-45, RJ-11, RS-232, eSATA, DisplayPort, DB25, etc), whereas the device controller is connected via a socket (connecting without soldering).

Sometimes, one device controller handles multiple I/O devices.



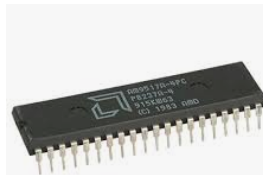
DMA (Direct Memory Access)

Motivation. Imagine you would like to download and save this slide on your hard-drive for future reference. By doing so, do you think it is necessary to involve the CPU for storing this pdf file in your hard-drive?

You would say that the data (pdf file) is received by your network card and is then transferred to the desired location on your hard drive.

Invoking the CPU to perform this transfer would certainly be an overkill. CPU's cycles would be wasted.

Now you could imagine, when multiple devices (or peripherals) are all attempting to access memory locations. **This is the reason for the requirement for a DMA controller** (Motorola MC6844P (left) and Intel 8237A-4 (right)).



DMA (Direct Memory Access)

Problem to solve: Large amount of data need to be transfered from main memory (i.e., RAM) to an I/O device or the reverse.

Solution 1. The use of polling:

- CPU periodically sends polls to I/O devices asking for any event.
- I/O device controllers respond with a positive acknowledgment.
- I/O starts sending data to CPU which then sends them to memory.

Solution 2. The use of interrupts:

- I/O devices request the CPU to perform an I/O operation with RAM.
- The CPU will have to wait for the device to read/write data.
- There is a speed mismatch and lot of CPU cycles are wasted.
- The CPU is not meant to wait but to constantly execute instructions.

Solution 3. Uses DMA. It allows I/O devices to directly access the memory without [involving, disturbing] the CPU (in particular, I/O devices that do large transfers, e.g., HDD, Graphical card, Network card, Sound card, etc).

DMA (Direct Memory Access)

A device controller receives data from a connected device and stores the data temporarily in some special purpose registers, known as buffer.

E.g., Your Ethernet network card would buffer incoming network traffic (packets received from the Internet).

Also each device has a dedicated driver, which is a software code that interfaces b/w the device controller and the OS.

The driver contains interrupt handlers that service the interrupts that are generated by that device controller.

Hence, when the driver for an I/O device is installed, interrupt handlers that service the interrupts of that I/O device get registered.

DMA (Direct Memory Access)

Because there is only one system bus (address, control, and data bus), technically, the DMA controller can operate in one of the following modes:

- ❶ **Burst mode.** The CPU grants DMA controller access over the system bus. DMA occupies the system bus for transferring a complete block of data (256 to 512 bytes) before releasing the bus back to the CPU.
 - CPU can only access its local cache (L1, L2, and L3).
 - CPU may remain Idle waiting for the bus to be released.
- ❷ **Cycle-stealing mode.** The CPU grants DMA controller access over the system bus. The bus is relinquished for a few clock cycles so that the CPU can perform other tasks (use cache) while the transfer of few bytes takes place. (sometime — Single-cycle mode)
 - CPU can only access its local cache (L1, L2, and L3).
 - CPU may not remain Idle for longer time.
 - DMAC keeps asking for the bus for each transfer.

DMA (Direct Memory Access)

Cache Coherency. This occurs when a DMA operation involves accessing a memory location in RAM which values have been updated by the CPU but are still stored on the CPU's cache (i.e., not yet written to memory).

The DMAC would then transfer data that is not up-to-date. To solve this problem there are two possible solutions:

- ❶ **Fully cache coherent system.** Handled at the hardware level:
 - If DMA writes fresh data to memory, cached data may get invalidated. CPU must read the data again from memory.
 - If DMA reads data that has not yet been written to memory (from cache), the cache must be “flushed” before data is read by DMA.
- ❷ **Not Fully cache coherent system.** Handled at the software level. The operating system will be required to decide if the cache should be flushed prior to a DMA operation, or invalidated to be read again.

DMA — Example

DMA (Direct Memory Access)

Example. DMA transfer from device into memory (RAM):

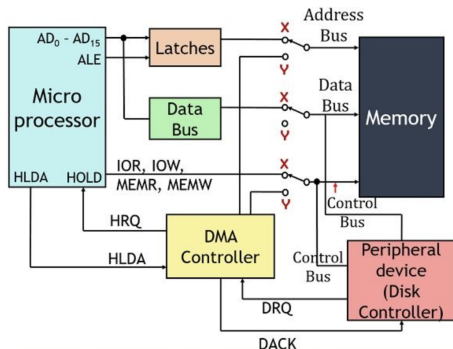
Device driver is instructed to transfer data from device to memory, e.g., transfer 256 bytes from disk at $@_d$ to RAM at $@_m$.

Device driver (thru ISR) instructs the device controller to transfer 256 bytes to central memory.

The device controller places a request for transfer to the DMA controller (i.e., DRQ — DMA Request).

DMA controller requests the system bus (i.e., HRQ — Hold Request).

The CPU then delegates the buses to the DMA controller (i.e., HLDA — Hold request Acknowledgement).



DMA (Direct Memory Access)

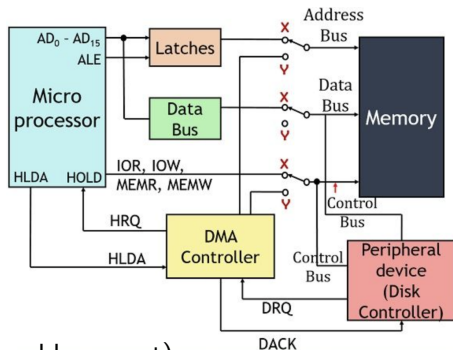
Example. DMA transfer from device into memory (RAM):

The DMA controller seizes the system bus (CPU is momentarily prevented from accessing the bus — hence the RAM). **At this time, the CPU can or cannot execute instructions.**

The DMA controller sets up the memory controller and informs the device controller to start the data transfer (DACK — DMA Acknowledgement).

Upon reception, the device controller starts sending each byte of data to the memory.

For each transferred byte, the DMA controller byte counter is decreased and its internal address register increased.



Example (Operating system password bypass via DMA)

This is how DMA can be used to exploit a computer system and bypass a software verification process.

● End